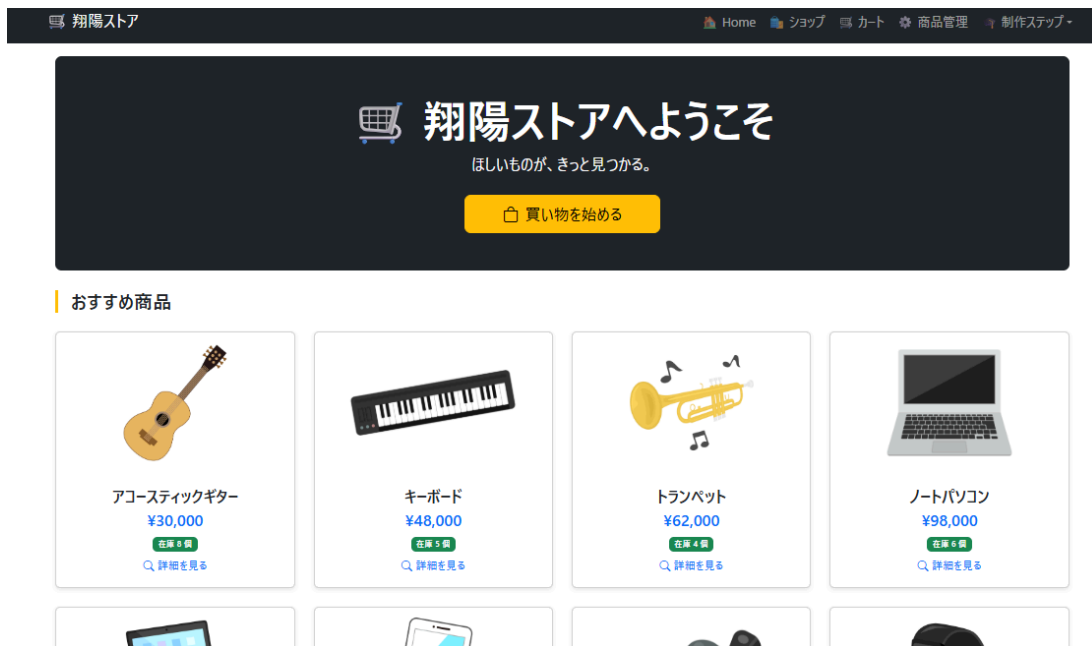


ショッピングサイトを作ろう！ ワークシート

ようこそ！ 皆さんは今日、これから 本物そっくりのインターネット通販サイトを 自分の手で完成させていきます。

難しいプログラムは、ほとんど先生が用意してあります。皆さんがやるのは「画面に何を出すか」を決めている部分を、自分で書き込むことです。



0. 準備 — まずは動かしてみよう

サイトを起動する

1. `shop-app` フォルダの中の `run.bat` をダブルクリックします。
2. 黒い画面(コマンドプロンプト)が開いて、こんな文字が出たら成功です。

ショッピングサイトが起動しました！

ブラウザで以下のURLを開いてください:

`http://localhost:3000`

3. ブラウザ(Edge や Chrome)を開いて、アドレスバーに `http://localhost:3000` と入力します。
4. お店のトップページが出てきたら準備完了！ 🎉

⚠️ 黒い画面は 閉じないで ください。閉じるとお店が止まります。

コードを編集する(VSCode)

- ファイルは **VSCode** で開いて編集します。
- 直したら 保存(**Ctrl + S**) → ブラウザを 更新(**F5**) すると、変化が画面に出ます。
- 「編集 → 保存 → ブラウザ更新」。この繰り返しが基本です。

この課題のルール

- 編集するのは、各ステップで指定された ファイルだけ です。
- コードの中の '**dummy**' や `_____` は「ここに自分で書き込む場所」という意味です。画面に「**dummy**」と出ていたり、絵が「こわれた絵」になっていたら、まだ完成していない印だよ。
- 間違えても、壊れません。何度でもやり直してOK！

EJS の基本ルール(これだけ覚えればOK)

このサイトの画面は **EJS** という仕組みでできています。覚えるのは2つだけ。

書き方	意味	例
<code><%= ○○ %></code>	○○の中身を 画面に表示する	<code><%= item.name %></code> → 商品名が出る
<code><% ~ %></code>	表示はしないで 計算や条件 をする	<code><% if (...) { %></code>

商品データは **item** という箱に入っていて、中身はこうなっています 📌

書き方	出てくるもの
<code>item.name</code>	商品名(例:ギター)
<code>item.price</code>	値段(例:30000)
<code>item.image</code>	画像のファイル名(例:guitar)
<code>item.stock</code>	在庫の数(例:10)
<code>item.code</code>	商品番号(例:1)
<code>item.description</code>	商品の説明文(詳細ページに出る)

Step 1 商品を登録しよう(増やす・減らす)

お店に並べる商品を登録します。やり方は2つ。

やり方① 管理画面のフォームから

URL <http://localhost:3000/step1> (ナビの  商品管理)を開きます。左のフォームに 商品名・価格・画像・商品説明 を入れて「追加する」を押すと、すぐに商品が増えます。要らない商品は右の一覧の  で消せます。

やり方② コードで最初の品ぞろえを決める(`db/seed.js`)

編集するファイル: `db/seed.js`

このファイルには「お店に最初から並べておく商品」が書いてあります。今は3つです。

```
db.run('INSERT INTO item (name, price, image, stock, description) VALUES ('ギター', 30000, 'guitar', 10, '初心者にもやさしいギター。')');
```

1行が「商品1つ」です。(`'商品名'`, `値段`, `'画像名'`, `在庫数`, `'説明文'`) の順に書きます。

課題1-A: 商品を1つ増やす

下の `_____` を自分で決めて、新しい行を 1行追加 しよう。使える画像名の一覧は、このシートの一番下にあります。

```
db.run('INSERT INTO item (name, price, image, stock, description) VALUES ('_____', _____, '_____', 10, '_____)');
```

課題1-B: 商品を1つ減らす

要らない商品の行を 1行消して みよう。(行の先頭にカーソル → その行をまるごと削除)

✓ 確認の仕方(大事)

`seed.js` を変えたら、`run.bat` をいったん閉じて、`run_reset.bat` (先生が用意)で起動し直すと、変えたデータで作り直せます。

`seed.js` は「最初の1回」だけ読まれるので、普通の `run.bat` では反映されません。
(フォームからの追加は、その場ですぐ反映されます。)

トップページ(`/`)や  商品管理 の画面で、商品が増えた・減ったか確認しよう。

≡ 現在の商品一覧 (データベースの中身)					
コード	画像	商品名	価格	在庫	削除
1		アコースティックギター	30,000円	8個	消
2		キーボード	48,000円	5個	消
3		トランペット	62,000円	4個	消
4		ノートパソコン	98,000円	6個	消
5		タブレット	45,000円	10個	消
6		スマートフォン	89,000円	7個	消

Step 2 カードで並べて、詳細ページを作ろう(穴埋め)

編集するファイル: **views/mikansei/product_card.ejs** (商品カードの共通パーツ)

URL <http://localhost:3000/step2> は、商品を大きなカードで並べる画面です。カードの「画像」「値段」「在庫」を完成させよう。カードの「詳細を見る」を押すと 商品詳細ページ (</item/番号>) が開きます。

✨ 大事: このカードは「共通パーツ」です。ここを直すと、Step2 だけでなく本番のお店 (</shop>) やトップページのカードも同時に変わります。自分が作った画面が、そのままお店で使われるということ!

課題2-A: 商品の画像を出す

カードのいちばん上に出る商品の画像です。'dummy' を書きかえよう。(画像は **product_card.ejs** の中に 2か所 あるので、両方とも 直してね。)

```

```

💡 ヒント: 画像のファイル名は **item.image**。<%= item.image %> と書くと </images/guitar.png> のように本物の画像が出ます。(直す前は「こわれた絵」、直すと商品の写真に変わるよ。)

課題2-B: 値段を出す

```
<p class="card-text text-primary fs-5 fw-bold mb-1">
```

```
  ¥<%= 'dummy' %>
```

```
</p>
```

💡 ヒント: 値段は **item.price.toLocaleString()** を後ろに付けると **30,000** のように 3けたごとにカンマが入って見やすくなります。(**item.price.toLocaleString()** と書いてみよう)

課題2-C: 在庫の数を出す

```
<span class="badge bg-success">在庫 <%= 'dummy' %> 個</span>
```

✅ 確認

カードに商品の画像・¥マーク付きの値段・在庫が出て、「詳細を見る」で詳細ページが開けば成功!



Step 3 🛒 「カートに入れる」ボタンを完成させよう(穴埋め)

編集するファイル: **views/mikansei/product_card.ejs** (Step2 と同じ共通パーツ)

URL <http://localhost:3000/step3> には「カートに入れる」ボタンがあります。でも今は 行き先も・どの商品か・色も 決まっていないので、押しても動きません(色もついていません)。このボタンを3つの穴埋めで完成させよう。

✨ Step2 と同じカードのパーツです。ボタンの部分(`showCart` が `true` のとき出る所)を直すと、本番のお店(`/shop`)の「カートに入れる」ボタンも同時に直ります。

完成させるのは、この1行の中にある 3つの '`dummy`' です 📍

```
<a href="<%= 'dummy' %>/<%= 'dummy' %>" class="btn <%= 'dummy' %> w-100">
  <i class="bi bi-cart-plus me-1"></i>カートに入れる
</a>
```

課題3-A: ボタンの「行き先」を入れる(1つ目の '`dummy`')

1つ目の '`dummy`' を `addurl` に書きかえよう(クォート ' ' は付けない)。

```
<a href="<%= addurl %>/<%= 'dummy' %>" ...>
```

💡 ヒント: カートに入れるときの行き先(URL)は `addurl` という箱に入っています。これは Step3 では `/cart/add`、本番ショップでは `/shop/add` に 自動で切り替わります。だから皆さんは `addurl` と書くだけでOK!

課題3-B: どの商品かを渡す(2つ目の '`dummy`')

2つ目の '`dummy`' を商品番号 `item.code` に書きかえよう。

```
<a href="<%= addurl %>/<%= item.code %>" ...>
```

💡 ヒント: これでリンク先が `/cart/add/3` のようになり、「〇番の商品をカートに入れて」と サーバーにお願いできます。

課題3-C: ボタンの色を決める(3つ目の 'dummy')

3つ目の 'dummy' (class="btn ..." の中) を、好きな色のクラスに書きかえよう。

```
<a ... class="btn btn-primary w-100">
```

💡 ヒント: 色は クラス名 で決まります。btn-primary=青 / btn-success=緑 / btn-danger=赤 / btn-warning=黄。'dummy' のままだと色のつかない地味なボタンのままだよ。

✅ 確認

ボタンに色がついて、押すとカート画面(Step4)に商品が入ればOK!

🛒 ショッピングカート

画像	商品名	価格	数量	小計	削除
	アコースティックギター	30,000円	1	30,000円	
	トランペット	62,000円	1	62,000円	
合計金額				92,000円	

🛒 購入する (合計 92,000円)

Step 4 🛒 カートを確認して「買う」(穴埋め+プログラム)

編集するファイル: **views/mikansei/cart_table.ejs** (カート表)、
views/mikansei/buy_button.ejs (購入ボタン)、**db/database.js** (お店の頭脳)

URL <http://localhost:3000/step4> はカートの中身を確認して購入する画面です。「小計(その商品1種類の合計)」を計算して出し、「購入する」ボタンを完成させ、さらに「購入したら在庫を減らす」プログラムを完成させよう。

✨ このカート表のパーツを直すと、本番のお店のカート(/cart)も同時に変わります。

課題4-A: 小計を計算して出す(画面)

ファイル: **views/mikansei/cart_table.ejs**

```
<td class="text-end align-middle fw-bold">
```

```
<%= 'dummy' %>円
```

```
</td>
```

💡 ヒント: 小計は「数量 × 値段」。cart.count(数量)と cart.price(値段)をかけ算します。
(cart.count * cart.price).toLocaleString() と書いてみよう。

課題4-B:「購入する」ボタンの送信先を入れる(画面)

ファイル: **views/mikansei/buy_button.ejs**

「購入する」ボタンも、今は送信先が決まっていないので押しても買えません。form の action の 'dummy' を buyurl に書きかえよう(クォート ' ' は付けない)。

```
<form action="<%= buyurl %>" method="POST">
```

💡 ヒント: 送信先は buyurl という箱に入っています。Step4 では /purchase、本番ショップの カートでは /buy に 自動で切り替わります。だから皆さんは buyurl と書くだけでOK! 'dummy' のままだと、押しても「買えない(404 エラー)」になります。

✨ この購入ボタンも共通パーツです。直すと、本番ショップのカート(/cart)の「購入する」ボタンも同時に直ります。

課題4-C:「購入する」ボタンの色を決める(画面)

ファイル: **views/mikansei/buy_button.ejs**

ボタンの色のクラス 'dummy' を、目立つ色に書きかえよう。

```
<button type="submit" class="btn btn-danger btn-lg px-5" ...>
```

💡 ヒント: btn-danger = 赤 / btn-primary = 青 / btn-success = 緑。「買う」ボタンは目立つ色がおすすめです。

課題4-D: 在庫を減らすプログラムを完成させる(本物のプログラム!)

ファイル: **db/database.js**

ここまでは「画面に何を出すか」を書いてきました。最後は お店の頭脳 の部分。「購入する」を押したとき、買った数だけ在庫を減らす計算 を自分で書きます。

purchase() (購入処理)の中に、こんな行があります 📌

```
// 新しい在庫 = 今の在庫 - 買った数
```

```
const newStock = item.stock; // ← 今は「今の在庫」のままなので、在庫が減らない!
```

この item.stock の部分を、「今の在庫 - 買った数」に書きかえよう。

```
const newStock = item.stock - item.count;
```

💡 ヒント:

- item.stock ... 今の在庫の数
- item.count ... カートに入っている(買う)数
- ひき算は記号 -(マイナス)を使います。

これで「〇個買ったから、在庫を〇個減らしてね」とデータベースにお願いできます。

✓ 確認

- カートに商品を入れて、小計と合計が正しく出ればOK！（合計は先生が計算してくれています）
- 課題4-B を直すと「購入する」ボタンが効くようになり、色(4-C)もつきます。
- 課題4-D を 直す前 は、「購入する」を押しても 在庫が減りません(ずっと10個のまま)。
- 課題4-D を 直したあとは、「購入する」を押すと Step2 のカードで 在庫の数が減ります(10個 → 9個 など)。自分が書いた計算で、お店の在庫が本当に動く！これが「データベースでお店の状態を管理する」仕組み。本物の通販サイトと同じです！

⚠ `db/database.js` を直したら、`run.bat` をいったん閉じて開き直すと確実に反映されます。

🛒 ショッピングカート

画像	商品名	価格	数量
	ワイヤレスイヤホン	12,800円	1
	スマートウォッチ	25,000円	1
			合計金額

🛒 購入する (合計 37,800円)

← 買い物続ける

Step 5 📦 「購入完了」の画面を作ろう

ファイル: `views/mikansei/complete.ejs`

URL `http://localhost:3000/step5`(Step4 で購入すると自動でここに来ます)。

買った商品の明細が表示されます。明細の表は共通パーツ `views/mikansei/order_table.ejs` です。本番の `/complete` と同じものを使っています。
「ご購入ありがとうございました」を表示させましょう！

- 「購入完了」のメッセージが出る
- 買った商品・数量・小計・合計が表に出る



ご購入ありがとうございました！
ご注文を承りました。

🛒 購入明細				
画像	商品名	数量	価格	小計
	ワイヤレスイヤホン	1	12,800円	12,800円
	スマートウォッチ	1	25,000円	25,000円
			合計	37,800円

🛒 もう一度買い物する

🏠 トップへ戻る

★ おかわり課題 — 商品詳細ページを完成させよう(時間が余ったら)

編集するファイル: `views/mikansei/item_detail.ejs` (商品詳細の共通パーツ)

カードの「詳細を見る」を押すと開く商品詳細ページ(`/item/番号`)にも、じつは Step2・Step3 と同じ穴埋め(`'dummy'`)が 6か所 残してあります。ここまでで覚えたことの復習です。同じ要領で完成させてみよう！

✨ この詳細パーツも共通部品です。直すと Step2 の「詳細」からも、本番のお店の商品詳細ページ(`/item/番号`)からも、同じきれいな画面が出ます。

課題D-A: 大きな商品画像を出す (Step2-A と同じ)

```

```

💡 ヒント: 画像のファイル名は `item.image`。

課題D-B: 値段を出す (Step2-B と同じ)

```
¥<%= 'dummy' %>
```

💡 ヒント: 値段は `item.price`。`item.price.toLocaleString()` でカンマ付きになるよ。

課題D-C: 在庫の数を出す (Step2-C と同じ)

```
<span class="badge bg-success fs-6">在庫 <%= 'dummy' %> 個</span>
```

💡 ヒント: 在庫の数は `item.stock`。

課題D-D: 「カートに入れる」の行き先を入れる (Step3-A と同じ)

```
<a href="<%= addurl %>/<%= 'dummy' %>" ...>
```

💡 ヒント: 1つ目の `'dummy'` を `addurl` に(クォートは付けない)。

課題D-E: 「カートに入れる」に商品番号を渡す (Step3-B と同じ)

```
<a href="<%= addurl %>/<%= item.code %>" ...>
```

💡 ヒント: 2つ目の `'dummy'` を商品番号 `item.code` に。

課題D-F: ボタンの色を決める (Step3-C と同じ)

```
<a ... class="btn btn-primary btn-lg px-5">
```

💡 ヒント: 3つ目の `'dummy'` を `btn-primary` などの色クラスに。

✅ 確認

商品カードの「詳細を見る」を押して、画像・値段・在庫がちゃんと出て、ボタンに色がついて、「カートに入れる」でその商品がカートに入れば完成！ 🎉

🌟 発展課題 — トップページを「自分のお店」にしよう

ここまでで仕組みが分かったね。最後に、お店のトップページ(/)を 自分だけのお店に改造 してみよう！

発展A: お店の名前を変える

ファイル: **views/index.ejs**

```
<h1 class="display-4 fw-bold">🛒 翔陽ストアへようこそ</h1>
```

```
<p class="fs-5 mb-4">ほしいものが、きっと見つかる。</p>
```

この **翔陽ストア** と キャッチコピーを、自分のお店の名前に変えよう。

ファイル: **views/mikansei/header.ejs** (画面の上のバー)

```
<a class="navbar-brand" href="/">🛒 翔陽ストア</a>
```

ここの店名も合わせて変えると、全部のページの上に表示されるよ。

発展B: 売る商品を自分で決める

Step 1 でやったように **db/seed.js** を編集して、自分のお店で売りたい商品をそろえよう。(5個以上を目指そう！)

発展C: 友達のお店をのぞいてみよう

同じネットワークなら、友達のサイトを操作したり、自分のサイトを友達に買い物してもらったりできます (先生の指示に従ってね)。

発展D: ナビ(上のバー)の色を変えよう

ファイル: **views/mikansei/header.ejs** (画面の上のバー)

お店のいちばん上にある黒いバー(ナビ)の 背景の色 と 文字の色 を、自分好みの色に変えてみよう。色 は クラス名 を書きかえるだけで変わります (CSSは書かなくてOK)。

今はこうなっています 📌

```
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
```

- 背景の色 ... **bg-dark** の部分を **bg-〇〇** に変える。
- 文字の色 ... **navbar-dark** (白っぽい字) か **navbar-light** (黒っぽい字) で切りかえる。背景が明るい色のときは **navbar-light** にすると文字が見えやすいよ。

たとえば「黄色の背景 + 黒い文字」にするなら 📌

```
<nav class="navbar navbar-expand-lg navbar-light bg-warning">
```

💡 使える色クラス(背景は `bg-○○`、文字は各リンクの `text-○○`):

色クラス(○○の部分)	だいたいの色
<code>primary</code>	青
<code>secondary</code>	灰
<code>success</code>	緑
<code>danger</code>	赤
<code>warning</code>	黄
<code>info</code>	水色
<code>light</code>	白に近い薄灰
<code>dark</code>	黒に近い濃灰

文字を1つずつ派手な色にしたいときは、メニューのリンクに `text-○○` を足してもOK👉

`<a class="nav-link text-danger" href="/">🏠 Home</a>`

💡 もっと奇抜な色(蛍光ピンクなど)にしたい人は、`public/css/styles.css` のいちばん下に `.navbar { background-color: #ff00cc !important; }` のように自分で色を書く方法もあるよ(先生に聞いてみよう)。

📷 (ここに自分のお店のスクショ)

✅ ふりかえりチェックリスト

- ☐ `run.bat` でサイトを起動できた
- ☐ Step1: 商品を登録した(増やす／減らす)
- ☐ Step2: カードに画像・値段・在庫を出し、詳細ページを開けた
- ☐ Step3: カートに入れるボタンを完成させた(行き先・商品番号・色)
- ☐ Step4-A: 小計を出した
- ☐ Step4-B / 4-C: 購入するボタンの送信先と色を入れて、購入できた
- ☐ Step4-D: 在庫を減らすプログラムを書いて、在庫が減るのを確認した
- ☐ Step5: 購入完了画面が表示された
- ☐ おかわり: 商品詳細ページ(D-A～D-F)を完成させた
- ☐ 発展: トップページを自分のお店にした(ナビの色も変えた)



使える画像の名前一覧

`db/seed.js` や課題で画像を指定するときは、この名前を使います。(例: ギターなら `guitar`)

楽器: guitar, midi, electone, piano, trumpet, sax, horn, dram, timpani

電子機器: notepc, tablet, phone, earphone, drone, watch, meter

乗り物: bike1, bike2, kickboard

ファッション: jacket, jeans1, jeans2, hat1, visor, shoes1, shoes2, shose3, shose4, sandal1, sandal2, crocs1, crocs2, megane1, megane2, megane3, bag, ruck

キッチン・家電: microwave, suihanki, washing, soujiki, fryingpan, pot, silverpot, handmixer, konro

文房具: ballpen1, ballpen2, hasami, stapler, tape, bond, board

スポーツ: baseball, tennis

その他: candle1, candle2, thermometer, bad

困ったら、まず「保存したかな?」「黒い画面は開いてるかな?」を確認しよう。それでもダメなら先生を呼んでね! 🙋